Same-Dataset Accuracy Does Not Transfer: Cross-Dataset Evaluation of Intrusion Detectors on WSN-DS, CICIoT2023, and Edge-IIoTset

Riteesh Thiruveedhula
Dept. of CSE-AIML
K G Reddy College of Engineering and Technology, Hyderabad, India
riteesh.vnsp@gmail.com

Velagala Sai Tarun Reddy
Dept. of CSE-AIML
K G Reddy College of Engineering and Technology, Hyderabad, India
saitarunrdy@gmail.com

Mogili Vikas Goud
Dept. of CSE-AIML
K G Reddy College of Engineering and Technology, Hyderabad, India

Shaik Gafar
Dept. of CSE-AIML
K G Reddy College of Engineering and Technology, Hyderabad, India

***Abstract*—Intrusion detection models are usually trained and tested on a single benchmark, which says little about how they behave on traffic from another network. We measure this on three public wireless and Internet of Things security datasets, WSN-DS, CICIoT2023, and Edge-IIoTset, after mapping their attack labels to a shared five-class taxonomy. The models are Random Forest and XGBoost baselines, a hybrid convolutional-recurrent model with self-attention, and a domain adversarial neural network that reuses the hybrid encoder and is trained on pooled data from two of the datasets. Each model is scored on held-out test data from the datasets it was trained on. Random Forest and the hybrid model are also trained on each dataset and tested on the other two. Same-dataset accuracy ranges from 91.9% to 99.6%. Cross-dataset accuracy is at most 67.6% for Random Forest and 80.7% for the hybrid model, and every Random Forest transfer score falls below the share of the largest class in the target test set. On Edge-IIoTset, Random Forest reaches 92.1% accuracy but only 65.9% macro-F1. The adversarial model scores 1.1 to 1.3 points below the hybrid model trained on each dataset separately. Scoring protocols differ between models, and we list each difference. For these datasets, same-dataset accuracy alone is not sufficient evidence, and a transfer claim needs matched scoring protocols, per-class metrics, and repeated runs.**

***Index Terms*—cross-dataset evaluation, domain adaptation, Internet of Things, intrusion detection, wireless sensor networks**

I. Introduction

Wireless sensor networks and Internet of Things (IoT) deployments expose many resource-constrained devices to denial-of-service, spoofing, and reconnaissance attacks. Studies of learning-based intrusion detection systems (IDSs) report very high accuracy on individual benchmarks. Such a score measures how well a model recognizes attacks produced by the capture process, feature extractor, and label set it was trained on. A deployed detector sees a different network and a different measurement pipeline. Sommer and Paxson argued that machine learning for intrusion detection calls for especially careful evaluation [1].

We measure the gap between same-dataset and cross-dataset results on three public benchmarks that differ in network layer and measurement level: WSN-DS [2], CICIoT2023 [3], and Edge-IIoTset [4]. We ask (i) how same-dataset results compare with results when a model trained on one dataset is tested on another; (ii) whether tree ensembles and a hybrid network behave differently under the two evaluations, where the hybrid combines a convolutional neural network (CNN), a bidirectional long short-term memory (BiLSTM) network, and attention; and (iii) whether a domain adversarial neural network (DANN) [5] trained on two pooled datasets changes same-dataset results relative to per-dataset training. Our contributions are:

- a shared five-class label mapping and two feature tracks for the three datasets, documented in Tables II and III;
- same-dataset results for Random Forest (RF) [6], XGBoost [7], the hybrid model, and DANN, plus confusion-matrix metrics for the two tree baselines, which show that RF's 92.1% accuracy on Edge-IIoTset comes with a macro-F1 of 65.9%;
- transfer matrices for RF and the hybrid model, in which every RF transfer accuracy is below the share of the largest class in the target test set;
- a list of the protocol differences between experiments (Table IV), which limit what the comparisons can show.

II. Related Work

WSN-DS covers four denial-of-service attacks on wireless sensor networks running the low-energy adaptive clustering hierarchy (LEACH) protocol [2], [8]. CICIoT2023 contains 33 attacks in seven categories executed against 105 IoT devices [3]. Edge-IIoTset was generated on an IoT and industrial IoT (IIoT) testbed and contains 14 attacks in five categories [4].

Surveys of deep learning for intrusion detection catalog architectures, datasets, and comparisons [9]. RF [6] and gradient-

TABLE I
BASELINE TEST SETS (20% STRATIFIED SPLIT)

Dataset	Classes	Test size: class counts	Majority
WSN-DS	0,1,2,3	74,933: 68,014/1,328/4,929/662	90.8%
CICIoT2023	0,2,3,4	39,997: 934/421/38,307/335	95.8%
Edge-IIoTset	0,2,3,4	39,999: 29,121/22/6,092/4,764	72.8%

Counts follow the class order and come from the saved baseline confusion matrices. Majority: share of the largest class. The composition of the hybrid and DANN test sets was not saved.

boosted trees in XGBoost [7] are standard strong baselines on tabular network features. Hybrid networks combining convolutions, long short-term memory (LSTM) [10] in bidirectional form [11], and attention [12] are common in recent IDS work, for example on wireless IoT sensor networks [13]. Our hybrid model belongs to this family; we configured it ourselves and did not reproduce that work.

Cross-dataset studies of network intrusion detection report large drops when a model is tested on a dataset it was not trained on [14], [15], including macro-F1 losses of up to 30 points for federated models moved between Edge-IIoTset, CICIoT2023, and TII-SSRC-23 [16]. We add a wireless sensor network dataset and compare each RF transfer score with the target's majority-class share.

Each network intrusion detection dataset ships with its own feature set, which makes models hard to compare across datasets. Sarhan et al. address this by re-expressing datasets in a common NetFlow-based feature set [17]. Our behavioral track (Section IV-A) is a cruder approach that maps feature slots by hand, and it has not been validated as a common representation. Domain adaptation theory bounds target error by source error plus a domain divergence [18], and Pan and Yang survey transfer learning [19]. DANN connects a domain classifier to a shared feature extractor through a gradient reversal layer, so training pushes the extractor toward domain-invariant features [5], [20]. Ganin et al. formulated it for labeled source data and unlabeled target data [5]. Our DANN experiment uses labels from both domains (Section IV-C).

III. DATASETS AND LABEL MAPPING

We obtained the three datasets as tabular copies from Kaggle. The WSN-DS and CICIoT2023 listings are not under the original authors' names, and we did not verify their contents against the original releases. The archived loader draws a class-stratified sample of about 200,000 records per dataset (at least 200 per class, the rest in proportion to class frequency). For CICIoT2023 and Edge-IIoTset, the saved baseline test sets (Table I) fit this cap, but the WSN-DS test set shows that WSN-DS was used in full. We split each dataset 70/10/20 into training, validation, and test partitions, stratified by class, and the archived code uses seed 42 for sampling and splitting.

We map raw labels to a five-class project taxonomy (Table II) and discard records whose label is not in the mapping. We built the taxonomy by judgment and did not validate it. Class 1 ("Jamming") holds only WSN-DS time-division multiple access (TDMA) scheduling attacks and does not occur in the other datasets. Class 2 ("Spoofing") puts blackhole and grayhole routing misbehavior in WSN-DS together with ARP spoofing, DNS spoofing, and man-in-the-middle traffic in the other two. These are different mechanisms, and the datasets observe them through different features. Since the label sets differ, cross-dataset scoring must decide what to do with a class that only one side has (Section IV-C).

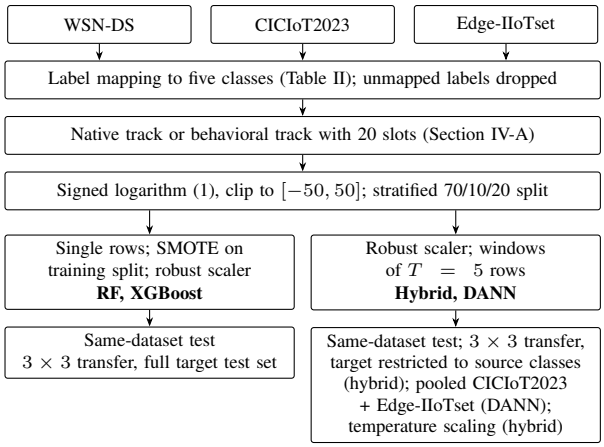


Fig. 1. Experimental pipeline. The baselines and deep models share the first three stages and differ after the split (Table IV). Each dataset is scaled with its own robust scaler.

IV. METHODS

A. Feature Tracks and Preprocessing

The *behavioral track* defines 20 named slots (volume, rate, send/receive ratio, acknowledgement ratio, and so on) and fills each one with a hand-written expression over each dataset's own columns. Only the slot names are shared. As Table III shows, the "energy" slot holds expended energy in WSN-DS, total packet size in CICIoT2023, and a UDP stream index in Edge-IIoTset. Same-dataset evaluation uses a *native track*. For WSN-DS and CICIoT2023 it has the same 20 columns as the behavioral track. For Edge-IIoTset it has 35 raw protocol fields (e.g., TCP flags, sequence and acknowledgement values, and DNS, MQTT, and Modbus fields) that we chose for class discrimination.

Fig. 1 summarizes preprocessing. We pass each value through a signed logarithm,

$$\tilde{x} = \operatorname{sign}(x)\, \log\bigl(1 + |x|\bigr), \tag{1}$$

and clip it to $[-50, 50]$. Both steps act on single values, so applying them before the split leaks no information. After the stratified split, the RF and XGBoost runs apply the synthetic minority oversampling technique (SMOTE) [21] to the training partition. We then fit a robust scaler on the training partition (after oversampling, for the baselines) and apply it to all partitions, separately for each dataset. For the deep models, we cut each partition into windows of $T = 5$ consecutive rows and label each window by its last row. The split shuffles rows, so the rows in a window are usually not adjacent in the original capture.

TABLE II
LABEL MAPPING FROM DATASET ATTACK LABELS TO THE FIVE-CLASS TAXONOMY

Class	WSN-DS	CICIoT2023	Edge-IIoTset
0 Normal	Normal	BenignTraffic	Normal
1 Jamming[a]	TDMA	–	–
2 Spoofing	Blackhole, Grayhole	MITM-ArpSpoofing, DNS_Spoofing	MITM
3 DoS_Flood	Flooding	19 labels: DDoS-*, DoS-*, and Mirai flood variants (e.g., DDoS-SYN_Flood, DoS-UDP_Flood, Mirai-udpplain)	DDoS_UDP, DDoS_ICMP, DDoS_TCP, DDoS_HTTP
4 Intrusion_Probe	–	12 labels: Recon-* (four), VulnerabilityScan, CommandInjection, SqlInjection, XSS, BrowserHijacking, Backdoor_Malware, DictionaryBruteForce, Uploading_Attack	SQL_injection, Password, Vulnerability_scanner, Uploading, Backdoor, Port_Scanning, XSS, Ransomware, Fingerprinting

[a]The class keeps the name used in the project taxonomy although it contains only scheduling attacks. A dash means the dataset has no records mapped to that class.

TABLE III
EXAMPLES OF BEHAVIORAL FEATURE SLOT DEFINITIONS PER DATASET

Slot	WSN-DS	CICIoT2023	Edge-IIoTset
volume	`DATA_S`+`DATA_R`	`Tot sum`	`tcp.len`+`mqtt.len`
ack ratio	`ADV_R`/(`ADV_S`+1)	`ack_count`/`Number`	`tcp.flags.ack`
energy	`Expaned Energy`	`Tot size`	`udp.stream`
port entropy	`Rank`/(max `Rank`+1)	`Covariance`	|`tcp.srcport`−`tcp.dstport`|/(sum+1)
mqtt activity	constant 0	`DNS`+`DHCP`+`ARP`	`mqtt.len`+`mqtt.ver`

Entries are the dataset column names used in the slot expressions; spellings follow the datasets.

B. Models

1) Baselines: RF [6] is a strong baseline for tabular data that needs little tuning. We use the scikit-learn implementation [22] with 100 trees, a fixed random seed, and default settings otherwise. We include XGBoost [7] to see whether gradient boosting behaves differently from bagged trees. It uses 100 trees, maximum depth 6, learning rate 0.1, and a multiclass softmax-probability objective.

2) Hybrid CNN-BiLSTM-attention: This model, our own configuration of the hybrid family in [13], tests whether a learned representation transfers differently from trees. Two one-dimensional convolutions (kernel 3, 32 then 64 channels, batch normalization, ReLU, dropout 0.2) feed a two-layer bidirectional LSTM with 128 units per direction and dropout 0.3 between layers. Next come a single-head self-attention layer [12], $\mathrm{softmax}(QK^{\top}/\sqrt{256})\,V$ with Q, K, and V learned linear projections of the 256-dimensional outputs, an average over the five time steps, and a head $256 \to 128 \to C$ (ReLU, dropout 0.3), where C is the number of classes in the training data. Training uses AdamW [23] (learning rate 10^{-3}, weight decay 10^{-4}), batch size 256, and at most 20 epochs. The learning rate halves after three epochs without validation improvement, and early stopping has patience 7 on validation loss. The loss is cross-entropy with class weights $w_c = \min\{5, \max\{0.5,\, N/(C\,n_c)\}\}$, where n_c is the number of training windows of class c and N the total.

3) DANN: DANN [5] trains features that do not reveal which domain a sample came from, a property a cross-dataset detector would want. It reuses the hybrid encoder, whose 256-dimensional output feeds a threat head and, through a gradient reversal layer, a domain head ($256 \to 128 \to 2$). The loss is $\mathcal{L}_{\mathrm{threat}} + \lambda\,\mathcal{L}_{\mathrm{domain}}$ with $\lambda = 0.1$, a class-weighted threat loss, and an unweighted domain loss. The reversal strength is $\alpha(p) = 2/(1 + e^{-5p}) - 1$, where p is the fraction of adversarial epochs completed, so α reaches about 0.98 in the last one. Training runs eight threat-only epochs and then up to 25 adversarial epochs. It uses the hybrid's optimizer and schedule, with early stopping at patience 7 on the combined validation loss.

C. Evaluation Protocols

Same-dataset: we train each model on a dataset's training partition and score it on that dataset's held-out test partition. *Cross-dataset transfer:* a model trained on a source dataset is scored on the test partition of a target dataset, for all nine pairs, and the target test set is scaled with the target's own scaler. RF is scored on the full target test set, so target samples from classes the source never saw count as errors. For the hybrid model, the target test set is first restricted to classes present in the source dataset. The two protocols differ, so their accuracies should not be differenced directly. *DANN:* we train it on the pooled training partitions of CICIoT2023 and Edge-IIoTset (each scaled with its own scaler) with threat labels from both domains, and score it on each dataset's own test partition. This is not a held-out-domain transfer test, and unlike the original formulation [5], the target domain is labeled.

Metrics: we saved accuracy for every model. For RF and XGBoost we also saved weighted precision, recall, and F1, the false alarm rate (FAR, the fraction of Normal samples predicted as any attack class), and confusion matrices, from which we derive macro-F1 and balanced accuracy (mean per-class recall). For the hybrid model and DANN we saved accuracy and weighted F1, and for the transfer matrices only accuracy. To calibrate, we fit one temperature on the validation partition by minimizing negative log-likelihood [24] and com-

TABLE IV
PROTOCOL DIFFERENCES BETWEEN EXPERIMENTS

Aspect	RF / XGBoost	Hybrid, same-dataset	Hybrid, transfer	DANN (CIC + Edge)
Feature track	20-slot schema[a]	Native (20; 35 for Edge-IIoTset)	Behavioral (20 slots)	Behavioral (20 slots)
Edge-IIoTset file	Not recorded[a]	"ML" selected file	"DNN" selected file	"DNN" selected file
Input; balancing	One row; SMOTE	5-row window; class weights	5-row window; class weights	5-row window; class weights
Transfer scoring	Full target test set	Not applicable	Target restricted to source classes	Not a transfer test
Saved metrics	Accuracy, weighted P/R/F1, FAR, confusion matrix	Accuracy, weighted F1	Accuracy	Accuracy, weighted F1
Training	100 trees (XGBoost: depth 6, rate 0.1)	$\leq$20 epochs	$\leq$20 epochs	8 + $\leq$25 epochs, $\lambda = 0.1$

[a]The saved baseline results record neither the feature schema nor the input file. The archived baseline script applies the 20-slot schema to every dataset but is not consistent with the archived Edge-IIoTset loader, so the schema and file behind the saved Edge-IIoTset baseline numbers cannot be confirmed.

pute expected calibration error (ECE) over 15 bins on the test partition. We ran each configuration once, and the saved deep-learning results do not record the model-initialization seed. Table IV summarizes the differences.

V. EXPERIMENTAL RESULTS

A. Same-Dataset Results

RF and XGBoost lead the native-track hybrid model on WSN-DS and CICIoT2023 by 1.5 to 1.6 points (Table V). On Edge-IIoTset the native-track hybrid leads with 94.39%, against 92.06% for RF and 91.92% for XGBoost. Weighted F1 follows the same order wherever it was saved. DANN, trained on pooled CICIoT2023 and Edge-IIoTset data, scores 1.29 and 1.13 points below the per-dataset behavioral hybrid models, which use the same feature track and data files. We also tried a three-domain DANN that added WSN-DS. Project notes say it did not converge to a useful solution. We saved no metrics from that run and make no quantitative claim about it.

B. Metrics Derived From Confusion Matrices

Only RF and XGBoost have saved confusion matrices (Table VII). Both reach macro-F1 near 97% on WSN-DS, with FAR below 0.4%. On CICIoT2023 their accuracy is 99.4%, yet macro-F1 is only 88.3% (RF) and 88.9% (XGBoost), and FAR is 8.24% and 9.31%. Edge-IIoTset has the widest gap, with accuracy above 91.9% and macro-F1 of 65.9% and 65.5%. On Edge-IIoTset, RF detects all 22 test samples of class 2 but also assigns 2,298 of the 6,092 class 3 samples (37.7%) to class 2, so class 2 has a precision of 0.9% (Fig. 2). RF's recall on CICIoT2023 is near 100% for class 3, which makes up 95.8% of the test set, and 91.8%, 79.3%, and 82.1% for the smaller classes.

C. Cross-Dataset Transfer

Off the diagonal of the transfer matrices (Table VI), RF accuracy ranges from 0.18% to 67.64% and hybrid accuracy from 0.30% to 80.70%. Between WSN-DS and CICIoT2023 both models score below 2% in either direction. From Edge-IIoTset to WSN-DS the hybrid model reaches 26.96%, against 1.90% for RF. Every off-diagonal RF accuracy is below the majority-class share of its target test set. The hybrid scores use masked targets: a model trained on WSN-DS is tested only on classes 0, 2, and 3, tests on WSN-DS leave out class 1, and pairs between CICIoT2023 and Edge-IIoTset need no mask. We did not save the composition of the masked test sets.

D. Calibration

The fitted temperatures for the native-track hybrid models are 1.040 (WSN-DS), 1.495 (CICIoT2023), and 1.077 (Edge-IIoTset). Temperature scaling raised ECE on all three datasets, in the same order: from 1.70% to 1.86%, from 0.23% to 0.43%, and from 1.78% to 2.04%.

VI. DISCUSSION AND LIMITATIONS

A. Same-Dataset Versus Cross-Dataset Accuracy

Every same-dataset result exceeds 91% accuracy. The largest off-diagonal entry is 80.7%, and most are far lower. This fits models that rely on signals specific to each dataset's capture process and feature definitions, but our experiments cannot separate four contributors: genuine distribution shift, behavioral slots that mean different things (Table III), classes that mean different things (Table II), and classes missing from the source or target. The taxonomy itself is coarse and unvalidated.

B. Accuracy Under Class Imbalance

The class distributions are skewed, so accuracy and macro-F1 diverge: for RF, 99.4% against 88.3% on CICIoT2023 and 92.1% against 65.9% on Edge-IIoTset. All six RF transfer accuracies are below what a constant prediction of the largest class would achieve. For the hybrid model we cannot check this, because we did not save the masked test-set composition. Suppose the masked Edge-IIoTset subset in the WSN-DS-to-Edge-IIoTset test resembles the baseline split, where Normal makes up 29,121 of the 35,235 samples in classes 0, 2, and 3 (82.6%). Then the reported 80.7% would be below always predicting Normal. We did not measure this, so it is only a conditional observation, and without per-class results that transfer score is not evidence of useful transfer.

C. Comparing Models

RF and XGBoost lead on WSN-DS and CICIoT2023 and are nearly indistinguishable. The native-track hybrid leads on Edge-IIoTset, yet it differs from the baselines in features, input

TABLE V
SAME-DATASET TEST ACCURACY AND WEIGHTED F1 (%), SINGLE RUN

	Accuracy					Weighted F1			
Dataset	***RF***	***XGB***	***Hybrid (native)***	***Hybrid (behav.)***	***DANN***	***RF***	***XGB***	***Hybrid (native)***	***DANN***
WSN-DS	99.60	99.56	98.06	97.79	–	99.60	99.57	98.15	–
CICIoT2023	99.42	99.44	97.80	97.79	96.50	99.42	99.44	97.59	96.03
Edge-IIoTset	92.06	91.92	94.39	97.42	96.29	94.25	94.01	94.65	96.32

XGB: XGBoost. "Hybrid (native)" and "Hybrid (behav.)" use the native and behavioral tracks; weighted F1 for the latter was not saved. A dash means the model was not trained on that dataset. Feature schemas and data files differ across columns (Table IV).

TABLE VI
CROSS-DATASET TRANSFER ACCURACY (%): ROWS ARE TRAINING DATASETS, COLUMNS ARE TEST DATASETS

(a) Random Forest, full target test set

	WSN-DS	**CICIoT2023**	**Edge-IIoTset**
WSN-DS	**99.60**	0.18	11.94
CICIoT2023	1.26	**99.42**	26.30
Edge-IIoTset	1.90	67.64	**92.06**
Majority share	90.8	95.8	72.8

(b) Hybrid, target restricted to source classes

	WSN-DS	**CICIoT2023**	**Edge-IIoTset**
WSN-DS	**97.79**	0.30	80.70
CICIoT2023	0.99	**97.79**	15.40
Edge-IIoTset	26.96	61.77	**97.42**
Majority share	not saved		

Bold entries are same-dataset results. "Majority share" is the largest class's share of the column dataset's RF baseline test set (Table I). The panels differ in class handling, feature tracks, and training (Table IV) and are not directly comparable.

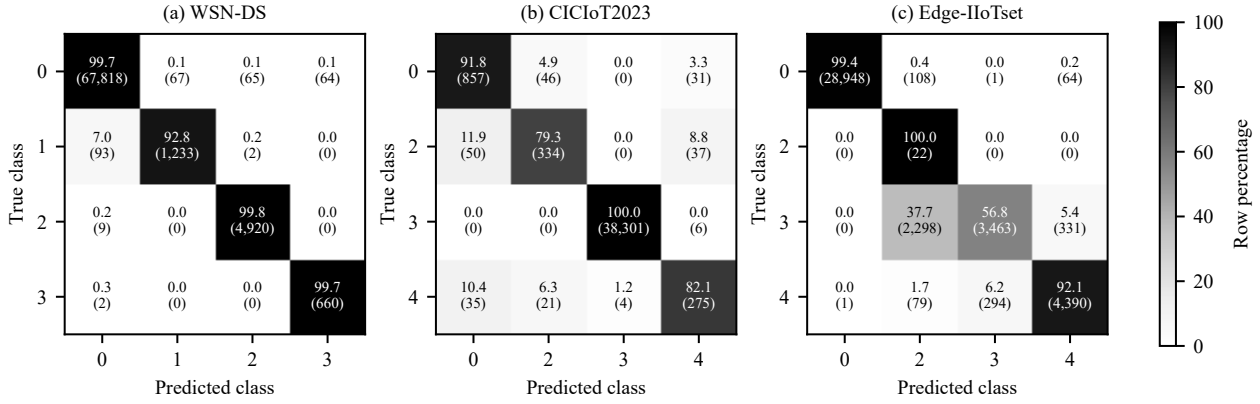


Fig. 2. Row-normalized confusion matrices of the Random Forest baseline on the held-out test sets: row percentage with the sample count in parentheses. Class identifiers follow Table II.

TABLE VII
CONFUSION-MATRIX-DERIVED METRICS FOR THE BASELINES (%)

Model	**Dataset**	**Acc.**	**W-F1**	**Macro-F1**[a]	**Bal. acc.**[a]	**FAR**
RF	WSN-DS	99.60	99.60	97.02	98.02	0.29
RF	CICIoT2023	99.42	99.42	88.26	88.29	8.24
RF	Edge-IIoTset	92.06	94.25	65.93	87.10	0.59
XGB	WSN-DS	99.56	99.57	97.09	98.06	0.33
XGB	CICIoT2023	99.44	99.44	88.86	89.42	9.31
XGB	Edge-IIoTset	91.92	94.01	65.51	86.93	0.58

[a]Derived from the saved confusion matrices. W-F1: weighted F1; XGB: XGBoost.

file, and class balancing (class weights instead of SMOTE), and its macro-F1 is unavailable. The 2.3-point gap therefore cannot be attributed to the architecture. We chose the 35 native Edge-IIoTset fields without recording the procedure, and if labels from the full dataset informed the choice, that result may be optimistic. In native mode the archived loader also reads the "ML" selected Edge-IIoTset file, and in the other modes the "DNN" file [4], so native and behavioral Edge-IIoTset results use different records. In the transfer tests, class handling confounds the comparison, and XGBoost and DANN have no transfer results. Scaling each target with its own scaler assumes unlabeled target data that a deployed detector may lack. Finally, windows come from a shuffled split, so the recurrent and attention layers see no real temporal structure, and the results say nothing about temporal modeling.

D. What the DANN Results Show

DANN's 96.5% and 96.3% show that one model trained on pooled data can nearly match per-dataset training on both datasets. Whether the adversarial term produced domain-invariant features is untested. Testing it would take a pooled control without gradient reversal trained for the same number of epochs, a held-out-domain test, and a record of domain-classifier accuracy. We have none of these. We also did not save the number of epochs run and did not tune λ or the schedule. Because DANN saw labels from both domains, the

result says nothing about the unsupervised-target setting it was designed for [5].

E. Calibration

Temperature scaling did not reduce ECE, but the unscaled values were already small, and differences of a few tenths of a point from a single run are hard to interpret. The CICIoT2023 temperature (1.495) is also close to the optimizer's starting value of 1.5, so incomplete optimization cannot be ruled out.

F. Reproducibility and Scope

Every number comes from a single run without variance estimates, so small differences (e.g., RF versus XGBoost) should not be treated as meaningful. The saved results came from earlier versions of the scripts than the archived ones. In the archived code, the loader caps WSN-DS at about 200,000 records, whereas the saved WSN-DS test set implies the full dataset. Its baseline script is inconsistent with its Edge-IIoTset loader, and its deep-learning script writes seed and ROC fields that the saved results lack. Although the native and behavioral WSN-DS hybrid runs use the same 20 columns, their accuracies differ (98.06% against 97.79%). We could not confirm that the tables can be regenerated exactly from the archived code. We did not save the areas under the receiver operating characteristic and precision-recall curves (ROC-AUC and PR-AUC), deep-model confusion matrices and per-class metrics, or cross-dataset F1. Project notes also mention an Edge-IIoTset run without the signed logarithm, but no result, code path, or configuration for it was saved, so we do not report it. The experiments use no live wireless or IoT traffic and measure neither detection latency nor mitigation.

VII. Conclusion

RF, XGBoost, a hybrid CNN-BiLSTM-attention model, and DANN all reach 91.9% to 99.6% accuracy on held-out test sets from WSN-DS, CICIoT2023, and Edge-IIoTset when trained on the same dataset. Accuracy on a different dataset is much lower, and for RF it is always below the target's majority-class share. For these datasets a single same-dataset accuracy is not sufficient evidence, and a transfer claim needs matched scoring protocols, per-class metrics, and variance estimates. The baselines lead on WSN-DS and CICIoT2023, and the hybrid model leads on Edge-IIoTset. Pooled DANN scored 1.1 to 1.3 points below the per-dataset behavioral hybrid models. We do not name a best model or attribute differences to architecture or adversarial training, because feature schemas, data files, and class handling differ, each result comes from a single run, and DANN has no control. Future work should extract common features from raw captures [17], score all models the same way, repeat runs over several seeds, report ROC-AUC and PR-AUC, add a pooled control for DANN, and test on a genuinely held-out target domain.

References

[1] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symp. Secur. Privacy*, Oakland, CA, USA, 2010, pp. 305–316.

[2] I. Almomani, B. Al-Kasasbeh, and M. Al-Akhras, "WSN-DS: A dataset for intrusion detection systems in wireless sensor networks," *J. Sensors*, vol. 2016, Art. no. 4731953, 2016, doi: 10.1155/2016/4731953.

[3] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, "CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment," *Sensors*, vol. 23, no. 13, Art. no. 5941, 2023, doi: 10.3390/s23135941.

[4] M. A. Ferrag, O. Friha, D. Hamouda, L. Maglaras, and H. Janicke, "Edge-IIoTset: A new comprehensive realistic cyber security dataset of IoT and IIoT applications for centralized and federated learning," *IEEE Access*, vol. 10, pp. 40281–40306, 2022, doi: 10.1109/ACCESS.2022.3165809.

[5] Y. Ganin *et al.*, "Domain-adversarial training of neural networks," *J. Mach. Learn. Res.*, vol. 17, no. 59, pp. 1–35, 2016.

[6] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001.

[7] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[8] W. R. Heinzelman, A. P. Chandrakasan, and H. Balakrishnan, "Energy-efficient communication protocol for wireless microsensor networks," in *Proc. 33rd Annu. Hawaii Int. Conf. Syst. Sci.*, 2000, doi: 10.1109/HICSS.2000.926982.

[9] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *J. Inf. Secur. Appl.*, vol. 50, Art. no. 102419, 2020.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, Nov. 1997.

[12] A. Vaswani *et al.*, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 5998–6008.

[13] P. Phalaagae, A. M. Zungeru, A. Yahya, B. Sigweni, and S. Rajalakshmi, "A hybrid CNN-LSTM model with attention mechanism for improved intrusion detection in wireless IoT sensor networks," *IEEE Access*, vol. 13, pp. 57322–57341, 2025, doi: 10.1109/ACCESS.2025.3555861.

[14] L. D'hooge, M. Verkerken, T. Wauters, F. De Turck, and B. Volckaert, "Investigating generalized performance of data-constrained supervised machine learning models on novel, related samples in intrusion detection," *Sensors*, vol. 23, no. 4, Art. no. 1846, 2023, doi: 10.3390/s23041846.

[15] S. Layeghy and M. Portmann, "On generalisability of machine learning-based network intrusion detection systems," 2022, arXiv:2205.04112.

[16] M. A. Bilal, I. U. Islam, S. Idrees, M. Qasim, M. J. Khan, and J. Khan, "Dataset-centric evaluation of federated intrusion detection models in IoT networks," *Sci. Rep.*, vol. 16, Art. no. 2683, 2026, doi: 10.1038/s41598-025-32567-w.

[17] M. Sarhan, S. Layeghy, N. Moustafa, and M. Portmann, "NetFlow datasets for machine learning-based network intrusion detection systems," in *Big Data Technologies and Applications and Wireless Internet (BDTA/WiCON)*. Springer, 2021, doi: 10.1007/978-3-030-72802-1_9.

[18] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Mach. Learn.*, vol. 79, no. 1–2, pp. 151–175, 2010.

[19] S. J. Pan and Q. Yang, "A survey on transfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, no. 10, pp. 1345–1359, Oct. 2010.

[20] Y. Ganin and V. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *Proc. 32nd Int. Conf. Mach. Learn.*, 2015, pp. 1180–1189.

[21] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, 2002.

[22] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[23] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. Int. Conf. Learn. Represent.*, 2019.

[24] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Mach. Learn.*, 2017, pp. 1321–1330.